

6.5.3 RMS versus EDF

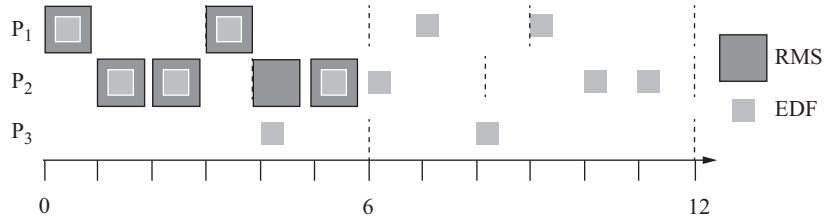
EDF can schedule some task sets that RMS cannot, as shown in the next example.

Example 6.4 RMS Versus EDF

Consider this example:

	C	T
P1	1	3
P2	2	4
P3	1	6

The hyperperiod of this task set is 12. Here is an attempt to construct RMS and EDF schedules for these tasks:



EDF successfully schedules the tasks with a utilization of 100%. In contrast, RMS misses P3's deadline at time 6.

6.5.4 Shared resources

A process may need to do more than read and write values to and from memory. For example, it may need to communicate with an I/O device. And it may use shared memory locations to communicate with other processes. When dealing with **shared resources**, special care must be taken.

Consider the case in which an I/O device has a flag that must be tested and modified by a process. Problems can arise when other processes may also want to access the device. If combinations of events from the two tasks operate on the device in the wrong order, we may create a **critical timing race** or **race condition** that causes erroneous operation. For example:

1. Task 1 reads the flag location and sees that it is 0.
2. Task 2 reads the flag location and sees that it is 0.

Race condition

3. Task 1 sets the flag location to 1 and writes data to the I/O device's data register.
4. Task 2 also sets the flag to 1 and writes its own data to the device data register, overwriting the data from task 1.

In this case, both devices thought they were able to write to the device, but the task 1's write was never completed because it was overridden by task 2.

Critical sections

To prevent this type of problem, we need to control the order in which some operations occur. For example, we need to be sure that a task finishes an I/O operation before allowing another task to start its own operation on that I/O device. We do so by enclosing sensitive sections of code in a **critical section** that executes without interruption.

Semaphores

We create a critical section using **semaphores**, which are a primitive provided by the operating system. The semaphore is used to guard a resource. (The term *semaphore* was derived from railroads, in which a shared section of track is guarded by signal flags that use semaphores to signal when it is safe to enter the track.) We start a critical section by calling a semaphore function that does not return until the resource is available. When we are done with the resource we use another semaphore function to release it. The semaphore names are, by tradition, `P()` to gain access to the protected resource and `V()` to release it.

```
/* some nonprotected operations here */
P(); /* wait for semaphore */
/* do protected work here */
V(); /* release semaphore */
```

This form of semaphore assumes that all system resources are guarded by the same `P()/V()` pair. Many operating systems also provide **named resources** that allow different resources to be grouped together. A call to `P()` or `V()` provides a resource name argument. Calls that refer to different resources may execute concurrently. Many operating systems also provide counting semaphores so that, for example, up to n tasks may pass the `P()` at any given time. If more tasks want to enter, they will wait until a task has released the resource.

Test-and-set

To implement `P()` and `V()`, the microprocessor bus must support an **atomic read/write** operation, which is available on a number of microprocessors. These types of instructions first read a location and then set it to a specified value, returning the result of the test. If the location was already set, then the additional set has no effect but the instruction returns a false result. If the location was not set, the instruction returns true and the location is in fact set. The bus supports this as an atomic operation that cannot be interrupted.

Programming Example 6.2 describes the ARM atomic read/write operation in more detail.

Programming Example 6.2 Compare-and-Swap Operations

The SWP (swap) instruction is used in the ARM to implement atomic compare-and-swap:

```
SWP    Rd, Rm, Rn
```

The SWP instruction takes three operands—the memory location pointed to by *Rn* is loaded and saved into *Rd*, and the value of *Rm* is then written into the location pointed to by *Rn*. When *Rd* and *Rn* are the same register, the instruction swaps the register's value and the value stored at the address pointed to by *Rd/Rn*. For example, the code sequence

```
        ADR r0, SEMAPHORE    ; get semaphore address
        LDR r1, #1
GETFLAG SWP r1, r1, [r0]      ; test-and-set the flag
        BNZ GETFLAG          ; no flag yet, try again
HASFLAG ...
```

first loads the constant 1 into *r1* and the address of the semaphore FLAG1 into register *r2*, then reads the semaphore into *r0* and writes the 1 value into the semaphore. The code then tests whether the semaphore fetched from memory is zero; if it was, the semaphore was not busy and we can enter the critical region that begins with the HASFLAG label. If the flag was nonzero, we loop back to try to get the flag once again.

The test-and-set allows us to implement semaphores. The *P()* operation uses a test-and-set to repeatedly test a location that holds a lock on the memory block. The *P()* operation does not exit until the lock is available; once it is available, the test-and-set automatically sets the lock. Once past the *P()* operation, the process can work on the protected memory block. The *V()* operation resets the lock, allowing other processes access to the region by using the *P()* function.

Critical sections pose some problems for real-time systems. Because the interrupt system is shut off during the critical section, the timer cannot interrupt and other processes cannot start to execute. The kernel may also have its own critical sections that keep interrupts from being serviced and other processes from executing.

6.5.5 Priority inversion

Shared resources cause a new and subtle scheduling problem: a low-priority process blocks execution of a higher-priority process by keeping hold of its resource, a phenomenon known as **priority inversion**. Example 6.5 illustrates the problem.

Example 6.5 Priority Inversion

A system with three processes: P1 has the highest priority, P3 has the lowest priority, and P2 has a priority in between that of P1 and P3. P1 and P3 both use the same shared resource. Processes become ready in this order:

- P3 becomes ready and enters its critical region, reserving the shared resource.
- P2 becomes ready and preempts P3.

- P1 becomes ready. It will preempt P2 and start to run but only until it reaches its critical section for the shared resource. At that point, it will stop executing.

For P1 to continue, P2 must completely finish, allowing P3 to resume and finish its critical section. Only when P3 is finished with its critical section can P1 resume.

Priority inheritance

The most common method for dealing with priority inversion is **priority inheritance**: promote the priority of any process when it requests a resource from the operating system. The priority of the process temporarily becomes higher than that of any other process that may use the resource. This ensures that the process will continue executing once it has the resource so that it can finish its work with the resource, return it to the operating system, and allow other processes to use it. Once the process is finished with the resource, its priority is demoted to its normal value.

6.5.6 Scheduling for low power

We can adapt RMS and EDF to take into account power consumption. While the race-to-dark case is more challenging, we have well-understood methods for using DVFS in conjunction with priority-based real-time scheduling [Qua07].

Using DVFS with EDF is relatively straightforward. The critical interval determines the worst case that must be handled. We first set the clock speed to meet the performance requirement in the critical interval. We then select the second-most critical interval and set the clock speed, continuing until the entire hyperperiod has been covered.

Using DVFS with RMS is, unfortunately, NP-complete. However, heuristics can be used to compute a good schedule that meets its deadlines and reduces power consumption.

6.5.7 A closer look at our modeling assumptions

Our analysis of RMS and EDF have made some strong assumptions. These assumptions have made the analyses much more tractable, but the predictions of analysis may not hold up in practice. Because a misprediction may cause a system to miss a critical deadline, it is important to at least understand the consequences of these assumptions.

Rate-monotonic scheduling assumes that there are no data dependencies between processes. Example 6.6 shows that knowledge of data dependencies can help use the CPU more efficiently.